OSSP SKILL WEEK 4

2520090105

Task 1

```
lokeshtella @LAPTOP-0PA61QEN:~$ nano task1_shell.c
lokeshtella @LAPTOP-0PA61QEN:~$ gcc task1_shell.c -o task1_shell
lokeshtella @LAPTOP-0PA61QEN:~$ ./task1_shell
myShell> ls
File1.txt     myshell          task1_shell.c  task5
File2.txt     process_fork     task2          task5.c
OSSP          process_fork.c   task2.c        week4_t1
fork_exec     shell.c          task3          week4_t1.c
fork_exec.c   task1            task3.c        week4_t2
hello         task1.c          task4          week4_t2.c
hello.c       task1_shell      task4.c        week4_t2.c.save
myShell> echo hello world
hello world
myShell> exit
Execution failed: No such file or directory
myShell> nano task1_shell.c
myShell> gcc task1_shell.c -o task1_shell
myShell> ./task1_shell
myShell> exit
Exiting myShell...
myShell>
```

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 1024
#define MAX_ARGS 64

int main() {
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    while (1) {
        printf("myShell> ");
        if (!fgets(input, sizeof(input), stdin)) break;

        // Remove trailing newline character
        input[strcspn(input, "\n")] = 0;

        // Check for empty input
        if (strlen(input) == 0) continue;

        // Tokenization
        int arg_count = 0;
        char *token = strtok(input, " \t");
        while (token != NULL && arg_count < MAX_ARGS - 1) {
            args[arg_count++] = token;
            token = strtok(NULL, " \t");
        }
        args[arg_count] = NULL;

        if (arg_count == 0) continue;

        // Handle built-in 'exit' command
        if (strcmp(args[0], "exit") == 0) {
            printf("Exiting myShell...\n");
            break;
        }

        // Process creation and execution
        pid_t pid = fork();
        if (pid == 0) {
            // Child process
            if (execvp(args[0], args) < 0) {
                perror("Execution failed");
                exit(1);
            }
        } else if (pid > 0) {
            // Parent process
            wait(pid, NULL, 0);
        } else {
            perror("Fork failed");
        }
    }
    return 0;
}
```

Task 2

```
root@LAPTOP-0PA61QEN:~# ls /home
lokeshtella
root@LAPTOP-0PA61QEN:~# su lokeshtella
lokeshtella@LAPTOP-0PA61QEN:/root$ ls
ls: cannot open directory '.': Permission denied
lokeshtella@LAPTOP-0PA61QEN:/root$ cd ~
lokeshtella@LAPTOP-0PA61QEN:~$ # nano task2_shell.c
lokeshtella@LAPTOP-0PA61QEN:~$ nano task2_shell.c
lokeshtella@LAPTOP-0PA61QEN:~$ gcc task2_shell.c -o task2_shell
lokeshtella@LAPTOP-0PA61QEN:~$ ./task2_shell
myShell> echo hello
Parsing Result:
Argument 1: echo
Argument 2: hello
Child PID: 657
hello
Command execution completed.
myShell>
```

GNU nano 7.2 task2_shell.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 1024
#define MAX_ARGS 64

int parse_single_quotes(char *input, char **args) {
    int arg_count = 0;
    char *ptr = input;

    while (*ptr != '\0') {
        // Skip leading whitespace
        while (*ptr == ' ' || *ptr == '\t') ptr++;
        if (*ptr == '\0') break;

        if (*ptr == '\'') {
            // Handle single-quoted argument
            ptr++; // Skip opening quote
            char *start = ptr;
            while (*ptr != '\'' && *ptr != '\0') ptr++;

            if (*ptr == '\0') {
                printf("Syntax Error: Unmatched single quote.\n");
                return -1;
            }

            *ptr = '\0'; // Null-terminate the string inside quotes
            args[arg_count++] = start;
            ptr++; // Skip closing quote
        } else {
            // Handle regular unquoted argument
            char *start = ptr;
            while (*ptr != ' ' && *ptr != '\t' && *ptr != '\'' && *ptr != '\0') ptr++;

            if (*ptr != '\0') {
                if (*ptr == '\'') {
                    // Stop at single quote without advancing ptr yet
                } else {
                    *ptr = '\0';
                    ptr++;
                }
            }
            args[arg_count++] = start;
        }
    }
    args[arg_count] = NULL;
    return arg_count;
}

int main() {
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    while (1) {
        printf("myShell> ");
        if (!fgets(input, sizeof(input), stdin)) break;

        input[strcspn(input, "\n")] = 0;
        if (strlen(input) == 0) continue;

        int count = parse_single_quotes(input, args);
        if (count <= 0) continue; // Skip on syntax error or no args

        // Display Parsing Results
        printf("Parsing Result:\n");
        for (int i = 0; i < count; i++) {
            printf("Argument %d: %s\n", i + 1, args[i]);
        }

        pid_t pid = fork();
        if (pid == 0) {
            // Child process execution
            execvp(args[0], args);
            perror("Execution failed");
            exit(1);
        } else if (pid > 0) {
            printf("Child PID: %d\n", pid);
            waitpid(pid, NULL, 0);
            printf("Command execution completed.\n");
        }
    }
```

Task 3:

```
lokeshtella @LAPTOP-0PA61QEN:~$ echo Hello
Hello
lokeshtella @LAPTOP-0PA61QEN:~$ echo 'Hello World'
Hello World
lokeshtella @LAPTOP-0PA61QEN:~$ echo '$USER'
$USER
lokeshtella @LAPTOP-0PA61QEN:~$ echo 'Hello World
> hello
```

Task 4:

```
root@LAPTOP-0PA61QEN:~# ls /home
lokeshtella
root@LAPTOP-0PA61QEN:~# su lokeshtella
lokeshtella @LAPTOP-0PA61QEN:/root$ ls
ls: cannot open directory '.': Permission denied
lokeshtella @LAPTOP-0PA61QEN:/root$ cd ~
lokeshtella @LAPTOP-0PA61QEN:~$ pwd
/home/lokeshtella
lokeshtella @LAPTOP-0PA61QEN:~$ echo "Hello $USER"
Hello lokeshtella
lokeshtella @LAPTOP-0PA61QEN:~$ echo "Hello     World"
Hello     World
lokeshtella @LAPTOP-0PA61QEN:~$ echo "Hello World"
Hello World
```